

Principles of Programming Languages

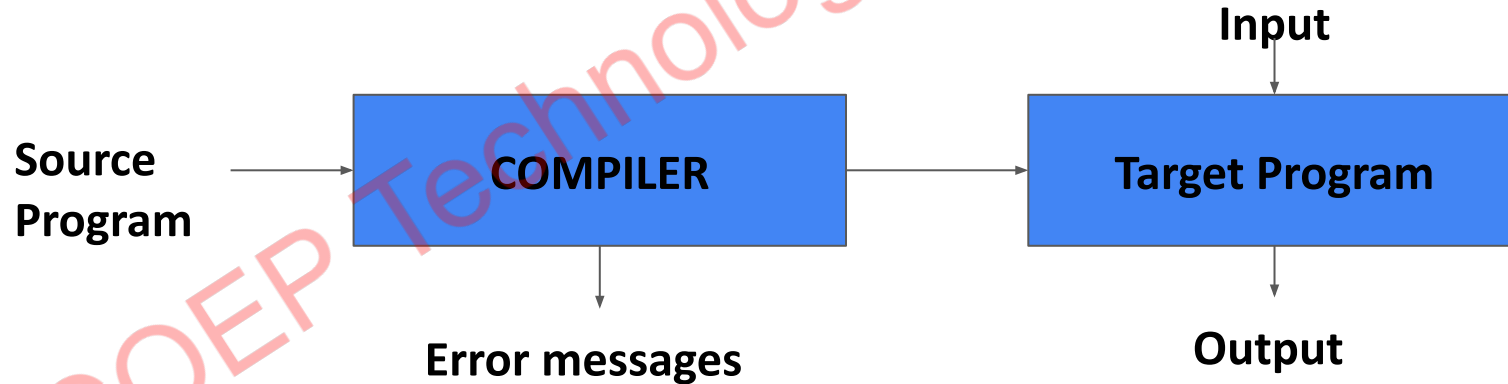
Compiler and Interpreters

Presented by: Kalyani Vidhate

COEP Technological University, Pune

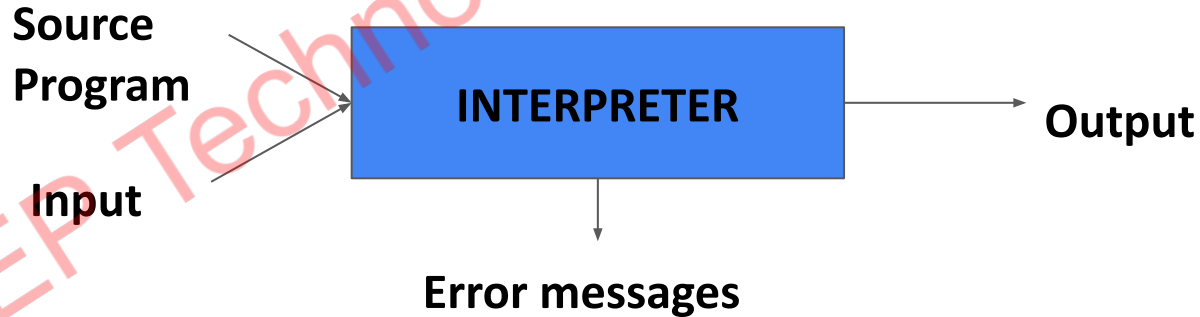
Compilers and Interpreters

Compilation : Translation of a program written in a source language into a semantically equivalent program written in a target language



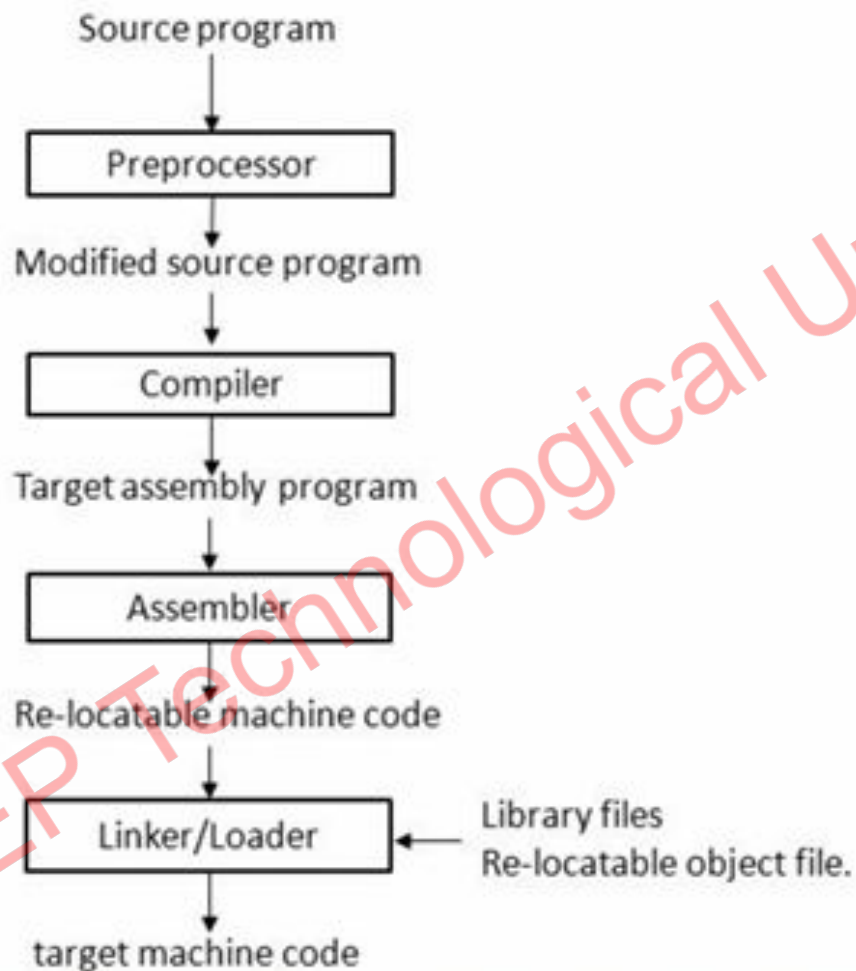
Compilers and Interpreters

Interpretation : Performing the operations implied by the source program

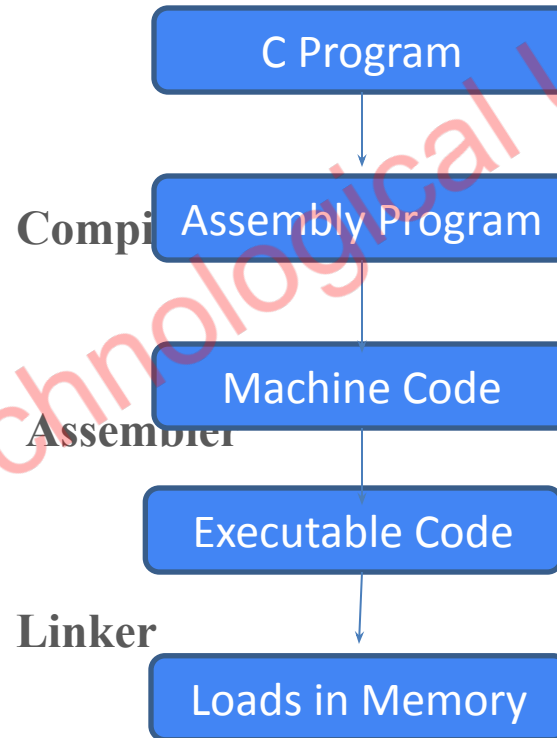


Feature	Compiler	Interpreter
Definition	A program that translates the entire source code into machine code before execution.	A program that translates and executes source code line-by-line at runtime.
Translation Method	Translates the entire program into machine code in one go.	Translates and executes the code line-by-line or statement-by-statement.
Output	Produces an executable file (.exe or binary).	No separate output file; executes the code directly without producing an executable.
Execution Speed	Generally faster, as the translated machine code is executed directly by the hardware.	Slower, as each line or statement is translated and executed sequentially.
Error Handling	All errors (syntax and some semantic) are reported after the entire code is analyzed.	Errors are detected at runtime and execution stops immediately upon encountering an error.
Dependency	Requires source code only once during compilation; the executable can be run independently.	Requires source code and interpreter for every execution.

Phases Involved	Lexical Analysis, Syntax Analysis, Semantic Analysis, Optimization, Code Generation, Assembly, and Linking.	Lexical Analysis, Syntax Analysis, Execution (no separate optimization or code generation phases).
Types of Errors Detected	Syntax errors, semantic errors, type errors are all detected during compilation.	Syntax errors are detected before execution; runtime errors are detected during execution.
Memory Usage	May use more memory as the entire program is loaded into memory during compilation.	Typically uses less memory since it handles one line of code at a time.
Example Languages	C, C++, Java (compiled to bytecode), Rust, Swift, Fortran.	Python, JavaScript, Ruby, PHP, Perl, MATLAB.
Advantages	Faster execution, optimized code, standalone executable files, good for performance-critical applications.	Easier debugging, platform independence, no separate compilation step, more flexible for rapid development.
Disadvantages	Slower compilation process, harder to debug due to separation of source code and executable.	Slower execution, requires interpreter every time, less optimized code, may need source code distribution.



How a C Language Program is executed?



Language Processors

- A source program **divided into modules** stored in separate files.
- The task of **collecting a source program** is sometimes entrusted to a separate program, called **preprocessor**.
- The **modified source program** is then **fed to a compiler**.
- The compiler produce an **assembly-language program** as its output.
- The assembly language is then processed by a program called an **assembler**.

Language Processors

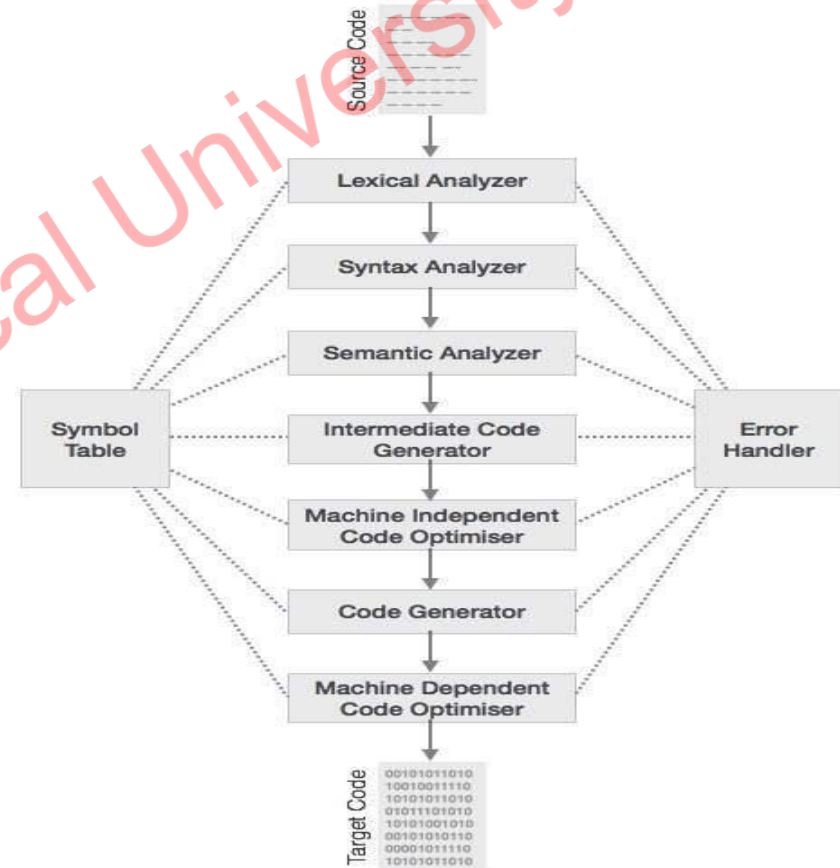
- An assembler produces **relocatable machine code** as its output.
- Large programs are often compiled in pieces, so the relocatable machine code may have to be linked together with other relocatable **object files** and **library files** into the code that actually runs on the machine.
- The **linker** resolves (decides) **external memory addresses**, where the code in one file may refer to location in another file.
- The **loader** then puts together all of the **executable object files** into memory for execution.

Phases of Compiler

COEP Technological University

PHASES OF COMPILER

- Compilation process is a sequence of various phases.
- Each phase takes input from previous phase.
- Feeds its output to the next phase.



Syntax and Semantics

Syntax

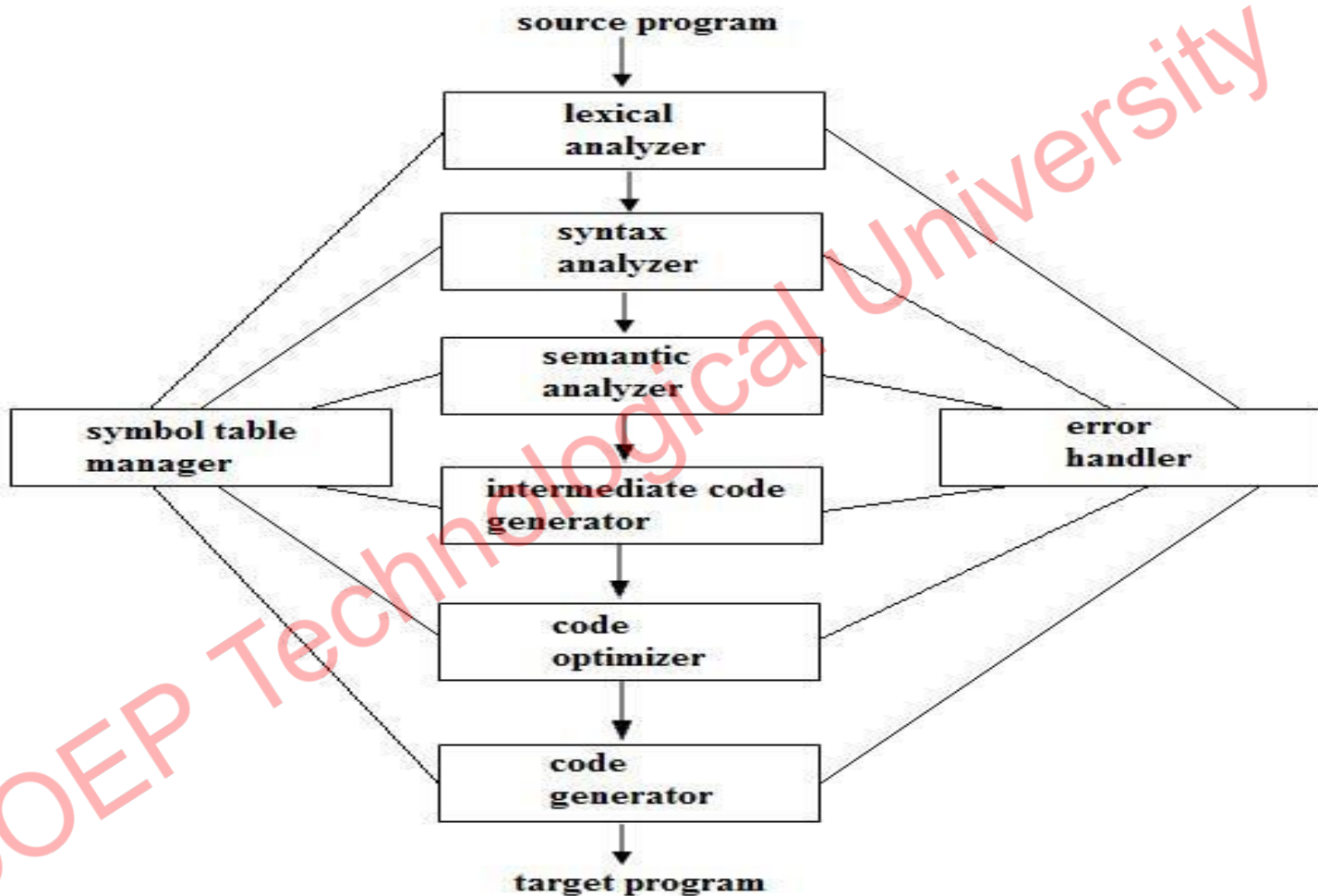
- **Definition:** Syntax refers to the rules that define **the structure of valid statements and expressions** in a programming language. It's analogous to grammar in natural languages, determining how symbols and keywords can be combined.

Semantics

- **Definition:** Semantics refers to the **meaning of the constructs** within a programming language. It defines what the **statements and expressions do when executed**.

Types of Semantics:

- **Operational Semantics:** Describes the behavior of a program in terms of the **operations performed during execution**. Often represented as state transitions of an abstract machine.
Example: The operational semantics of the expression $5 + 3$ can be described by stating that the $+$ operation takes two numbers as input and produces their sum as output.
- **Denotational Semantics:** Maps language constructs to mathematical objects. Focuses on the **abstract meaning** rather than the operational process.
Example: The denotational semantics of the expression $5 + 3$ can be defined as the function $\text{add}(5, 3)$, where add is a function that returns the sum of its arguments.
- **Axiomatic Semantics:** Uses logical assertions to describe the behavior of programs. Focuses on **preconditions and**



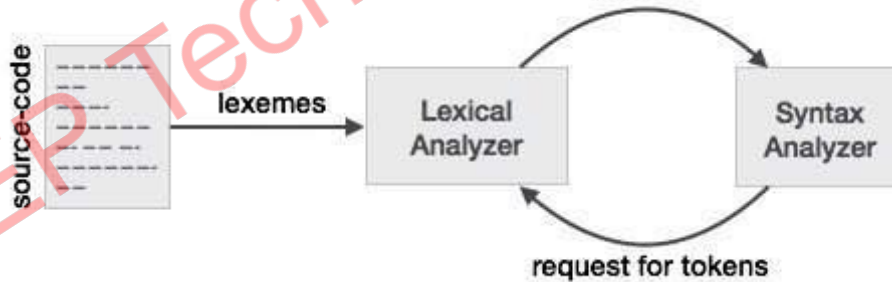
Phases of compiler

LEXICAL ANALYSIS

- First Phase of compiler.
- Breaks the source code to tokens.

WHAT IS A TOKEN?

A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming languages.



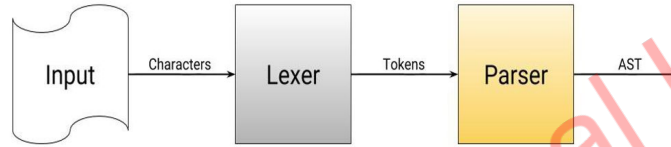
- **Lexical Analysis**

Lexical analysis is the first phase of a compiler. It takes the modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code. If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax

- **Tokens**
analyzer. It reads character streams from the source code, checks for legal tokens, and lexemes are said to be a sequence of characters (alphanumeric) in a token. There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern. A pattern explains what can be a token, and these patterns are defined by means of regular expressions. In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuation symbols can be considered as tokens.

- **Parsers**

In the syntax analysis phase, a compiler verifies whether or not the tokens generated by the lexical analyzer are grouped according to the syntactic rules of the language. This is done by a parser. The parser obtains a string of tokens from the lexical analyzer and verifies it against the syntactic rules of the language. It detects and reports any syntax errors and produces a parse tree.



- **CONTEXT FREE GRAMMAR**

A lexical analyzer can identify tokens with the help of regular expressions and pattern rules. But a lexical analyzer cannot check the syntax of a given sentence due to the limitations of the regular expressions. Regular expressions cannot check balancing tokens, such as parenthesis. Therefore, this phase uses context-free grammar (CFG).

Context-free grammar G can be defined by four tuples as:

$$G = (V, T, P, S)$$

Where,

G is the grammar, which consists of a **set of the production rule**. It is used to generate the string of a language.

T is the final **set of a terminal symbol**. It is denoted by lower case letters.

V is the final **set of a non-terminal symbol**. It is denoted by capital letters.

P is a **set of production rules**, which is used for replacing non-terminal symbols (on the left side of the production) in a string with other terminal or nonterminal symbols (on the right side of the production).

S is the **start symbol** which is used to derive the string.

We can derive the string by repeatedly replacing a non-terminal by the right-hand side of the production until all